ConfigMap or a Secret. I'll describe that strategy further in the next section of the chapter. Here we'll focus on the features offered by Spring Cloud Kubernetes and its subprojects.

Spring Cloud Kubernetes (https://spring.io/projects/spring-cloud-kubernetes) is an exciting project that provides Spring Boot integration with the Kubernetes API. Its original goal was to make it easier to transition from a microservices architecture based on Spring Cloud to Kubernetes. It provides an implementation for standard Spring Cloud interfaces used for service discovery and load balancing to integrate with Kubernetes, and it adds support for loading configuration from ConfigMaps and Secrets.

If you work on a greenfield project, you don't need Spring Cloud Kubernetes. Kubernetes provides service discovery and load balancing natively, as you experienced in chapter 7. Furthermore, Spring Boot supports configuration via ConfigMaps and Secrets natively, so there's no need for Spring Cloud Kubernetes, even in this case.

When migrating a brownfield project to Kubernetes, and it uses libraries like Spring Cloud Netflix Eureka for service discovery and Spring Cloud Netflix Ribbon or Spring Cloud Load Balancer for load balancing, you might use Spring Cloud Kubernetes for a smoother transition. However, I would recommend refactoring your code to leverage the native service discovery and load-balancing features from Kubernetes rather than adding Spring Cloud Kubernetes to your project.

The main reason why I recommend not using Spring Cloud Kubernetes in standard applications is that it requires access to the Kubernetes API Server to manage Pods, Services, ConfigMaps, and Secrets. Besides the security concerns related to granting applications access to the Kubernetes internal objects, it would also couple the applications to Kubernetes unnecessarily and affect the maintainability of the solution.